

Техническое задание: API-сервис «Офисный Обед»

1. Общее описание проекта

Цель проекта — разработать RESTful API для сервиса совместного заказа еды внутри закрытых групп (офисов, коворкингов, студенческих комнат). Сервис позволяет одному пользователю открыть «сессию сбора заказов» из конкретного ресторана, а остальным участникам — добавлять свои блюда в общую корзину до наступления контрольного времени.

2. Архитектура базы данных (Сущности и связи)

2.1. Таблица users (Пользователи)

- id (Integer, Primary Key)
- username (String, Unique, Not Null) — имя или никнейм пользователя.
- email (String, Unique, Not Null) — электронная почта.

2.2. Таблица sessions (Сессии заказов)

- id (Integer, Primary Key)
- creator_id (Integer, ForeignKey -> users.id) — кто организовал сбор.
- restaurant_name (String, Not Null) — название заведения.
- status (Enum / String, Default: active) — статусы: active (сбор), ordered (заказ отправлен в ресторан), delivered (еда приехала), cancelled (отменен).
- created_at (DateTime, Default: current time) — время создания.
- deadline (DateTime, Not Null) — время, до которого можно добавлять позиции.

2.3. Таблица menu_items (Элементы меню)

- id (Integer, Primary Key)
- session_id (Integer, ForeignKey -> sessions.id) — меню привязывается к конкретной сессии (чтобы избежать необходимости делать глобальный справочник ресторанов).
- name (String, Not Null) — название блюда (например, "Пицца Маргарита").
- price (Numeric / Integer, Not Null) — цена в копейках или рублях (целым числом для избежания проблем с float).

2.4. Таблица `order_items` (Элементы заказа / Корзина)

- `id` (Integer, Primary Key)
- `user_id` (Integer, ForeignKey -> `users.id`) — кто заказывает.
- `menu_item_id` (Integer, ForeignKey -> `menu_items.id`) — что заказывает.
- `quantity` (Integer, Default: 1) — количество порций.

3. Требования к API и Эндпоинтам

Интерфейс должен быть разделен на логические группы (Tags в Swagger).

3.1. Управление пользователями (`/users`)

- `POST /users` — Регистрация нового пользователя.
- `GET /users` — Получение списка всех пользователей (для выбора, кто подключается к заказу).

3.2. Управление сессиями заказа (`/sessions`)

- `POST /sessions` — Создание новой сессии.
 - *Бизнес-логика:* Принимает `creator_id`, `restaurant_name` и `deadline`. `deadline` должен быть строго в будущем времени.
- `GET /sessions` — Получение списка всех активных (`active`) сессий.
- `GET /sessions/{session_id}` — Детальная информация о конкретной сессии, включая список доступного меню.
- `PATCH /sessions/{session_id}/status` — Изменение статуса сессии организатором (Перевод в `ordered` или `delivered`).

3.3. Управление меню сессии (`/sessions/{session_id}/menu`)

- `POST /sessions/{session_id}/menu` — Добавление позиций меню в сессию. Принимает массив объектов (название, цена). Можно вызывать несколько раз.
 - *Бизнес-логика:* Добавление доступно только если статус сессии `active` и текущее время меньше `deadline`. В противном случае возвращать ошибку 400 Bad Request.

3.4. Формирование корзины заказа (`/orders`)

- `POST /orders` — Добавление блюда в свой заказ. Принимает `user_id`, `menu_item_id`, `quantity`.

- *Бизнес-логика*: Проверить, что сессия, к которой относится блюдо, активна и дедлайн не вышел. Если пользователь добавляет то же блюдо повторно, quantity должно увеличиваться.
- DELETE /orders/{order_item_id} — Удаление позиции из своего заказа (или уменьшение количества).

3.5. Агрегационные отчеты

- GET /sessions/{session_id}/summary — Получение сводного отчета по всему общему заказу.
 - **Что должен возвращать эндпоинт:**
 1. Общую стоимость всего заказа (сумма всех позиций с учетом их количества).
 2. Сгруппированный список для отправки в ресторан: какое блюдо и в каком суммарном количестве нужно заказать (например: *"Пицца Маргарита — 5 шт, Кола — 3 шт"*).
 3. Пользовательский сплит-чек: список участников и сколько каждый из них должен перевести организатору (с детализацией их личных заказов).

4. Обработка ошибок и Валидация

Задействовать механизмы HTTPException для обработки следующих кейсов:

1. **404 Not Found**: Запрос несуществующего пользователя, сессии или блюда.
2. **400 Bad Request**:
 - Попытка добавить блюдо в заказ, если deadline сессии уже наступил.
 - Попытка перевести сессию в статус ordered, если в ней еще нет ни одного заказа.
3. **422 Unprocessable Entity**: Автоматически обрабатывается Pydantic'ом (например, если передана отрицательная цена или количество блюд ≤ 0).

5. План выполнения работы

- **Этап 1**: Настройка окружения, инициализация SQLAlchemy (создание асинхронного движка AsyncSession).
- **Этап 2**: Написание ORM-моделей (models.py) и генерация базовых Pydantic-схем для валидации (schemas.py).
- **Этап 3**: Реализация базового CRUD для пользователей и создания сессий.

- **Этап 4:** Реализация логики добавления блюд и проверки дедлайнов (бизнес-логика в роутах).
- **Этап 5:** Написание сложных SQL-запросов с использованием группировок (`func.sum`, `func.count`) для получения финального отчета (`summary`).